

Subject: SAST Summer Training 2026, Agent Harness
Date: from 2026/10/10 to 2026/10/11

Contents

Architecture Overview and Entry Flow	1
架构总图	1
链路总图	2
程序如何进入 Agent	3
关键类型	4

Architecture Overview and Entry Flow

源码快照说明

本节基于工作区保存的 2026 年 3 月 31 日公开 source map 源码快照，通过简化后的结构图、数据类型和教学化伪代码解释 Claude Code 的主要执行链路。

该快照不是 Anthropic 官方源码仓库，也不代表后续版本的稳定架构. 示例保留关键控制流，但会省略 UI、兼容性和部分横切分支；其中个别辅助函数名仅用于表达职责，不一定与源码完全同名。

架构总图

子系统与依赖关系

架构图展示 Claude Code 的整体子系统及其依赖关系，而不是展开单次任务的循环细节. 入口与界面层承接终端、SDK、IDE 和远程会话；应用编排层负责配置、命令、扩展和多 Agent 协作；核心运行时统一连接模型与工具；状态服务和可观测性贯穿任务生命周期。

The diagram illustrates the system architecture of Claude Code, organized into several layers and components:

- 入口与界面层 (Entry and Interface Layer):** Includes `main.tsx / Commander CLI` and `IDE / Remote / Bridge / Server`. It connects to `Link REPL / Screens / Components` and `Print / SDK / MCP Endpoint`.
- 应用编排层 (Application Orchestration Layer):** Contains `Setup / Config / Auth`, `Command Registry / Slash Commands`, `Plugins / Skills / MCP Connections`, and `Tasks / Agents / Coordinator`. It interacts with the `QueryEngine` and `Tool Pool`.
- 核心运行时 (Core Runtime):** Features `QueryEngine / query() / queryLoop()`, `Context / Prompt / Compact`, `Model API Client / Streaming / Recovery`, and `Tool Pool / Tool Orchestration`. It also includes `Schema / Hooks / Permission / Policy`.
- 状态与辅助服务 (State and Auxiliary Services):** Includes `Transcript / Session / Resume`, `Memory / File History / Plans`, and `Telemetry / Cost / Diagnostics`.
- 外部系统与依赖环境 (External Systems and Dependencies):** Includes `Anthropic / Model Provider`, `Shell / Files / Git / LSP / Web`, `MCP Servers`, and `IDE / Remote Services`.

Figure 1: Claude Code 架构总图

五个层次的职责

层次	子系统	职责
入口与界面层	CLI、Ink REPL、Print/SDK、IDE/Remote	承接终端、SDK、IDE 与远程会话，构造一次任务所需的输入。
应用编排层	Setup、Command Registry、Plugins/Skills/MCP、Tasks/Agents	完成启动装配，并管理确定性命令与扩展能力。
Agent 运行时	QueryEngine、Context/Prompt、Model Client、Tool Pool、Policy	统一连接模型与工具，是任务循环的执行核心。
状态与横切服务	Transcript/Session、Memory、Telemetry	持久化会话与记忆，记录成本、诊断和可观测性数据。
外部系统与宿主环境	Anthropic、Shell/Files/Git、MCP Servers、Remote	提供进程外部的真实能力，如模型、文件系统和远端服务。

链路总图

一次 Query 的时序关系

时序图强调消息的先后关系：模型只提出工具调用请求，真正的权限校验和执行由 Harness 完成。

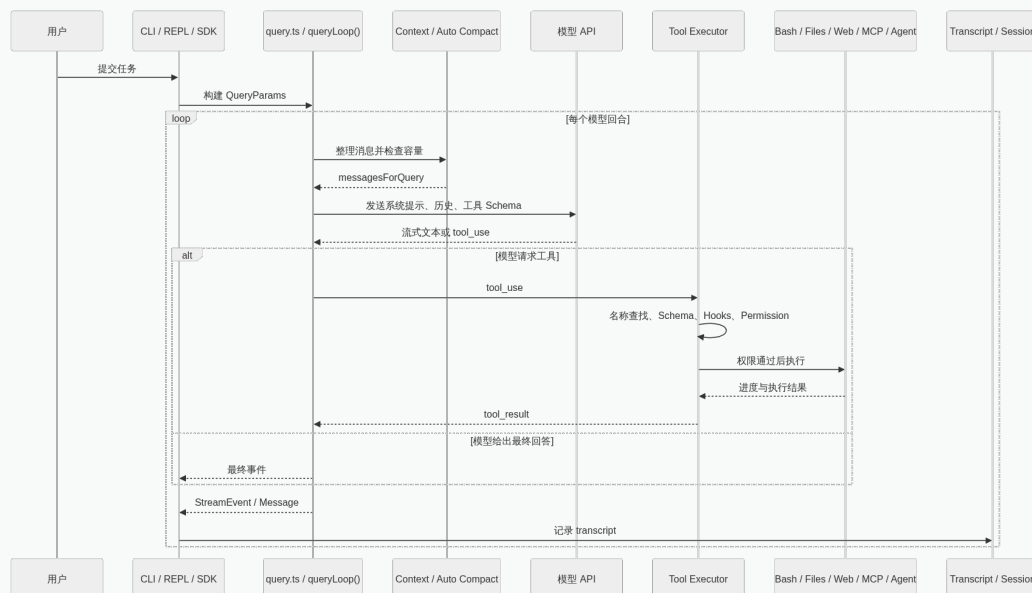


Figure 2: 一次 Query 的链路总图

程序如何进入 Agent

从进程启动到 Agent Loop

下面的调用树展示了 Claude CLI 从进程启动到进入 Agent Loop 的准备过程. `run()` 负责组装运行环境, `query()` 负责一次任务, `queryLoop()` 负责在任务内部持续决策和执行.

Claude CLI 进程启动

- 1. `main()`
 - 记录启动性能检查点
 - 设置进程级安全环境
 - 注册 `warning` / `exit` / `SIGINT` 处理器
 - 解析命令行参数
 - 调用 `run()`
- 2. `run()`
 - 2.1 读取运行配置
 - 用户配置
 - 项目配置
 - 环境变量
 - CLI 参数覆盖
 - 2.2 建立身份与会话
 - 认证信息
 - API Client
 - model 配置
 - session id / transcript
 - 2.3 加载项目上下文
 - cwd
 - CLAUDE.md
 - 项目设置
 - Git / 工作区信息
 - additional directories
 - 2.4 建立权限上下文
 - allow / deny rules
 - 用户确认回调
 - sandbox 配置
 - ToolUseContext
 - 2.5 注册和发现 Skill
 - `initBundledSkills()`
 - `getSkillDirCommands()`
 - 读取 SKILL.md
 - 转换为 Command / Skill 定义
 - 2.6 连接 MCP
 - `connectToServer()`
 - 选择 stdio / SSE / HTTP Transport
 - tools/list
 - `fetchToolsForClient()`
 - MCP Tool 转换为内部 Tool

- | - 2.7 组装 Tool Pool
 - | - getAllBaseTools()
 - | - 按运行模式过滤
 - | - 按 Agent 类型过滤
 - | - 应用 deny rules
 - | - 合并 MCP Tools
 - | - 排序、去重
- | - 2.8 选择交互外壳
 - | - REPL 交互模式
 - | - print mode
 - | - slash command
 - | - SDK / 受控入口
- 3. 用户提交任务
 - REPL 接收用户输入
 - 创建 user message
 - 合并已有 conversation messages
 - 收集 tools / model / permission context
 - 构造 QueryParams
- 4. query(QueryParams)
 - 创建命令生命周期记录
 - 调用 queryLoop()
 - 将内部事件作为 AsyncGenerator 向外 yield
 - 正常结束后标记命令 completed
- 5. queryLoop()
 - 进入 Agent Loop; 内部状态和执行转移见第 4 节

职责边界

最重要的职责边界是：

- main() / run(): 准备 Agent 能使用的环境；
- query(): 启动一次任务，并向 UI 输出事件；
- queryLoop(): 维护任务内部的循环状态。

关键类型

REPL

REPL (Read-Eval-Print Loop, 读取—求值—输出循环) 是一个长期运行的外层循环。每次用户提交普通文本，系统都会创建一次 Query；斜杠命令 (Slash Command) 则可能在本地直接处理，不进入模型。

下面的代码是教学化伪代码，重点展示输入分流、事件消费和界面恢复：

```
while (!shouldExit) {
  // readLine() 暂停当前循环，等待用户在终端中提交一行文本。
  const input = await readLine()
```

```

// 斜杠命令通常由 Harness 本地解析, 例如 /help、/compact 或 /exit.
if (input.startsWith('/')) {
    await runSlashCommand(input)
    continue
}

// 将本次输入与当前会话、工具列表、权限配置等组装为 QueryParams.
const queryParams = buildQueryParamsFromInput(input, session)

try {
    // query() 返回 AsyncGenerator. 每产生一个事件, UI 就可以立即处理.
    for await (const event of query(queryParams)) {
        // event.type 是可辨识联合类型的判别字段.
        switch (event.type) {
            case 'text_delta':
                terminal.write(event.text)
                break
            case 'tool_start':
                renderToolStart(event.toolName)
                break
            case 'tool_progress':
                renderToolProgress(event.toolUseId, event.text)
                break
            case 'permission_request':
                await renderPermissionPrompt(event)
                break
            case 'tool_result':
                renderToolResult(event)
                break
            case 'error':
                renderError(event.error)
                break
        }
    }
} finally {
    // 无论任务正常结束、抛错还是被取消, 都恢复输入提示符和光标状态.
    restorePromptAndCursor()
}
}

```

QueryParams

QueryParams 表示任务开始时传入的配置和依赖. 下面通过字段注释说明各部分的职责:

```

type QueryParams = {
    // 任务开始时已有的对话历史, 通常包含当前用户消息.
    messages: Message[]

    // 本次请求使用的系统提示, 定义角色、规则和总体行为边界.
    systemPrompt: SystemPrompt

    // 可由用户内容派生的补充信息, 例如显式附加的文件或说明.
    userContext: Record<string, string>
}

```

```

// Harness 注入的运行环境信息，例如工作目录、平台和日期。
systemContext: Record<string, string>

// 权限决策回调：工具需要确认时，通过它获得允许或拒绝结果。
canUseTool: CanUseToolFn

// 工具执行所需的动态环境，包括工具池、取消信号和应用状态。
toolUseContext: ToolUseContext

// 标记任务来源，例如交互终端、SDK 或子 Agent。
querySource: QuerySource

// 主模型不可用时允许切换到备用模型。
fallbackModel?: string

// 针对本次任务覆盖模型的最大输出 Token 数。
maxOutputTokensOverride?: number

// 限制 Agent Loop 的最大轮数，防止任务无限循环。
maxTurns?: number

// 为 true 时跳过缓存写入，常用于不适合持久化的请求。
skipCacheWrite?: boolean

// 限制任务可消耗的总预算；具体单位由实现约定。
taskBudget?: { total: number }

// 可注入的底层依赖，便于测试时替换模型、压缩器等实现。
deps?: QueryDeps
}

```

QueryEvent

QueryEvent 是 Query 对外输出的事件联合类型。调用方通过 type 判断事件种类，再读取相应字段：

```

type QueryEvent =
  // 模型新生成的一段文本，可直接追加到当前回答。
  | { type: 'text_delta'; text: string }
  // 工具开始事件；toolUseId 唯一标识本次调用。
  | { type: 'tool_start'; toolName: string; toolUseId: string }
  // 长时间运行的工具可以多次发出进度文本。
  | { type: 'tool_progress'; toolUseId: string; text: string }
  // 工具命中 ask 规则时，携带原始参数请求用户确认。
  | { type: 'permission_request'; toolUseId: string; input: unknown }
  // 工具最终结果；isError 区分正常观察与错误观察。
  | { type: 'tool_result'; toolUseId: string; content: string;
      isError: boolean }
  // 已组装完成的助手消息，可用于持久化完整对话结构。
  | { type: 'assistant_message'; message: Message }
  // 无法在正常事件流中恢复的任务级异常。
  | { type: 'error'; error: Error }

```

Async Generator: 基本概念

Async Generator（异步生成器）可以理解为一个“会异步执行，还能持续产生中间事件”的函数。普通 `async function` 只有在任务结束后才返回一个最终结果：

```
// 调用方会一直等待，期间无法从这个返回值获得中间进度。  
const result = await runTask()
```

Async Generator: 用 `yield` 产生事件

`async function*` 则可以在执行过程中通过 `yield` 多次产生事件：

```
async function* runTask() {  
  // 第一次 yield 立即把文本事件交给调用方，然后函数暂停在这里。  
  yield { type: 'text_delta', text: '开始分析' }  
  
  // 异步等待模型完成当前阶段：等待期间不会阻塞事件循环。  
  await runModel()  
  
  // 通知 UI 即将运行 Bash 工具。  
  yield { type: 'tool_start', toolName: 'Bash' }  
  
  // 等待外部命令结束并取得最终结果。  
  const result = await runCommand()  
  
  // 将命令结果作为最后一个事件交给调用方。  
  yield { type: 'tool_result', content: result }  
}
```

Async Generator: 按顺序消费事件

外层使用 `for await...of` 按事件到达顺序消费，这使得 UI 不需要等整个任务结束再渲染，而是收到一个事件就渲染一个事件：

```
for await (const event of runTask()) {  
  // 每收到一个事件就立即更新界面或状态。  
  render(event)  
}
```